Java I/O

12 August 2026 09:28

Java I/O (input and output) is used to read data from input sources and write data to output destinations.

What is Java I/O?
Java I/O refers to the process of handling input and output operations in Java applications. It is mainly used to read data from sources such as keyboard or files and write data to destinations like the console(terminal) or files.

Java uses the concept of streams to make input and output operations efficient. All Java I/O related classes are available in the **java.io** or **java.util or java.nio** package.

Java I/O is also widely used for file handling.

Core Concepts of Java I/O

Java I / O is a based on two main concepts:

1. **Streams:** A stream represents a continuous flow of data. In Java, streams are used to read or write data sequentially.
   There are two types of streams:
   - **Input stream:** Used to read data from a source.
   - **Output stream:** Used to write data to a destination.

   Streams are further classified into:
   i. **Byte Stream:** InputStream, OutputStream
   ii. **Character stream:** Reader, Writer

2. **Readers/Writers:** Readers and writers are specially designed for handling character-based data.

   Readers: Readers are used to read character data.
   Writers: Writers are used to write character data.

   They provide a convenient way to work with text data.

What is a Stream in Java?
A stream is a sequence of data bytes. It is called a stream because data flows continuously, similar to a stream of water.

Java automatically creates three standard streams which are connected to the console.

1) **System.out - Standard output stream.**
2) **System.err - Standard error stream.**
3) **System.in - Standard input stream.**

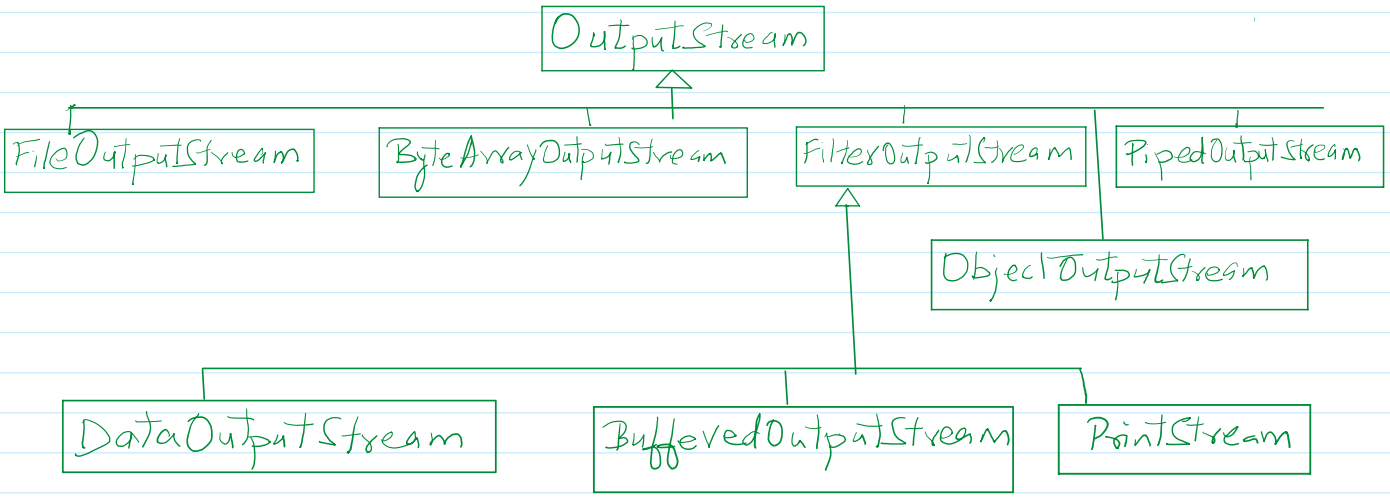
OutputStream class

Output stream class is an abstract class. It is the Super class of all the classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.

Methods of OutputStream Class

Method	Description
public void write(int) throws IOException	It is used to write a byte to the current output stream.
public void write(byte[])throws IOException	It is used to write an array of bytes to the current output stream.
public void flush() throws IOException	It flushes the current output stream.
public void close() throws IOException	It is used to close the current output stream.

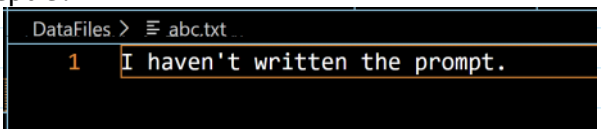
OutputStream Hierarchy




```java
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
public class OutputStreamExample {
    @SuppressWarnings("resource")
    public static void main(String[] args) {
        String path = "DataFiles/abc.txt";
        try (FileOutputStream fout = new FileOutputStream(path)){
            String content = new java.util.Scanner(System.in).nextLine();
            fout.write(content.getBytes());
            System.out.println("Data written Successfully");
        } catch (FileNotFoundException e) {
            e.printStackTrace();
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:



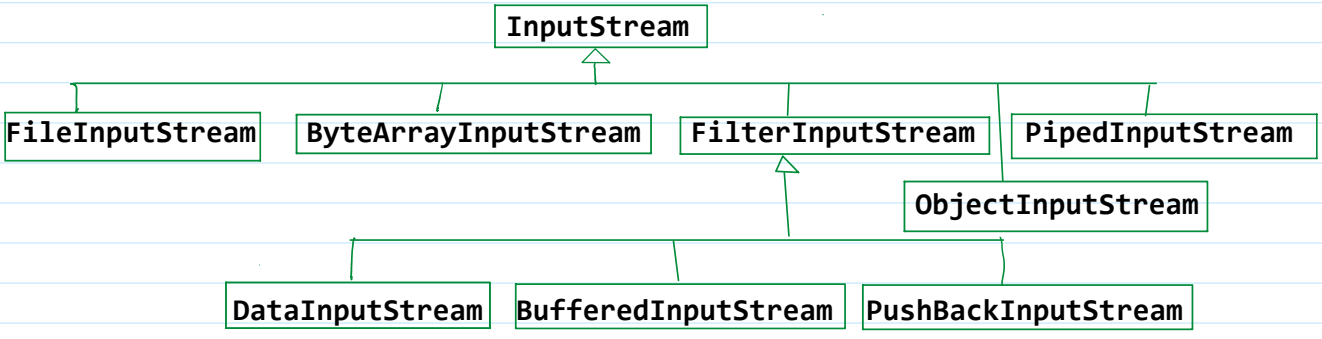
InputStream Class

Input stream class is an abstract class. It is the Super class of all classes representing an input stream of bytes.

Useful Methods of InputStream Class

Method	Description
public abstract int read() throws IOException	It reads the next byte of data from the input stream. It returns minus one (-1) at the end of the file.
public int available() throws IOException	It returns an estimate of the number of bytes that can be read from the current input stream.
public void close() throws IOException	It is used to close the current input stream.

InputStream Hierarchy



Example:

```
package FileHandling;
import java.io.FileInputStream;
public class InputStreamExample {
    public static void main(String[] args) {
        String path = "DataFiles/abc.txt";
        try (FileInputStream fis = new FileInputStream(path)) {
            int i;
```

```
            System.out.println("File content: ");
            while ((i = fis.read())!=-1) {
                System.out.print((char)i);
            }
            System.out.println();
        } catch (Exception e) {

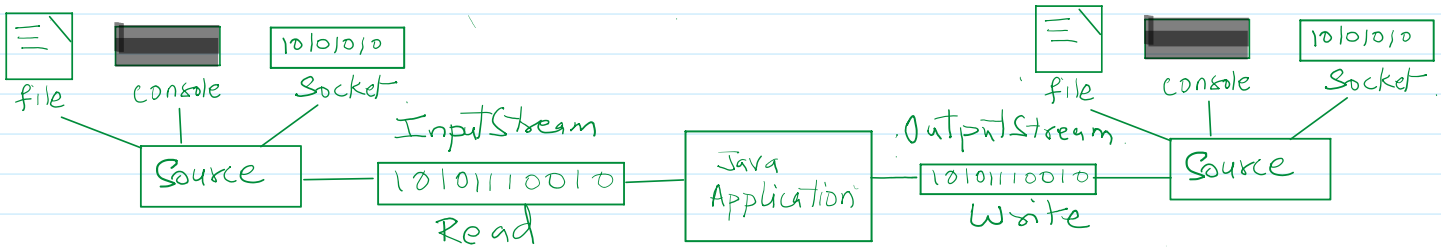
        }
    }
}
```

Output:

File content:
I haven't written the prompt.

OutputStream Vs. InputStream
The output stream and input stream classes are used to write data to and read the data from different sources and destinations.

An **OutputStream** is used by a Java application to write data to a destination. The destination can be a file, a byte array, a peripheral device, or a network socket. Whereas an **InputStream** is used by a Java applications to read data from a source. The source can be a file, a byte array, a peripheral device, or a network socket.



Java I/O Classes
Java provides a rich set of classes to perform input and output operations. These classes helps program read data from different sources and write data to various destinations.

- **InputStream and OutputStream**
- **Reader and Writer**
- **FileInputStream and FileOutputStream**
- **FileReader and FileWriter**
- **BufferedInputStream and BufferedOutputStream**
- **BufferedReader and BufferedWriter**

For Next classes
- **Text file Handling**

- **Binary file Handling(mp3 player)**